

Artificial Intelligence — Assignment Solutions

Water Jug Problem • Mars Rover PEAS • 8-Queens • OLA Cab Agent • UCS Logistics Network

Question 1: The Water Jug Problem

Given a 4-gallon jug and a 3-gallon jug, neither with markings, and a pump for filling, obtain exactly 2 gallons of water in the 4-gallon jug. Allowed actions: fill a jug completely, empty a jug completely on the ground, or pour from one jug into the other until the source is empty or the destination is full.

Problem Formulation

State: represented as a pair (x, y) where x = amount of water in the 4-gallon jug (0-4) and y = amount of water in the 3-gallon jug (0-3). Initial state: $(0, 0)$. Goal state: $(2, y)$ for any y .

Step	Action	4-gal jug	3-gal jug
0	Start	0	0
1	Fill the 4-gallon jug from the pump	4	0
2	Pour water from the 4-gallon jug into the 3-gallon jug	1	3
3	Empty the 3-gallon jug on the ground	1	0
4	Pour the remaining water from the 4-gallon jug into the 3-gallon jug	0	1
5	Fill the 4-gallon jug from the pump	4	1
6	Pour from the 4-gallon jug into the 3-gallon jug until it is full (3-gal jug needs 2 more, so 4-gal jug loses 2)	2	3

Result: After Step 6 the state is $(2, 3)$ — the 4-gallon jug contains exactly **2 gallons**, which is the goal. This sequence corresponds to a path found in the water-jug state space by uninformed search (e.g. BFS), where each edge is a fill / empty / pour operator.

Question 2: Mars Rover — PEAS Analysis of an Intelligent Agent

i. Percepts

- Stereo/panoramic camera images of the surrounding terrain
- Spectrometer and X-ray/laser readings of rock and soil composition
- Position and orientation data from GPS-like localisation / inertial sensors
- Obstacle and hazard data from LIDAR, sonar or stereo-vision range sensors
- Wheel odometry, tilt and accelerometer readings (slope, slippage)
- Battery / solar-panel power level and temperature readings
- Commands and mission updates transmitted from mission control on Earth

ii. Environment Characterisation

Property	Characterisation	Reason
Observability	Partially observable	Sensors see only nearby terrain; the full Martian surface is never fully known.
Number of agents	Single-agent (occasionally multi-agent with other rovers/orbiters)	Agent with multiple rovers/orbiters in pursuit of its own goals.
Determinism	Stochastic	Wheel slippage, dust storms and terrain give uncertain action outcomes.
Episodic vs Sequential	Sequential	Current actions (e.g. driving, drilling) affect future states and options.
Static vs Dynamic	Dynamic (semi-dynamic)	Weather, lighting and dust conditions change while the rover deliberates.
Discrete vs Continuous	Continuous	Position, movement and sensor readings are continuous-valued.
Known vs Unknown	Unknown	The rover does not have a complete a-priori model of Mars terrain.

iii. Actions

- Move forward / backward, turn left / right, climb or avoid obstacles
- Extend, position and operate the robotic arm
- Drill into rock / scoop soil to collect a sample
- Place a sample in an onboard instrument for analysis (spectrometer, chromatograph, microscope)
- Capture images (panoramic, close-up, stereo) of terrain or samples
- Transmit collected data / images to Earth via orbiter or direct link
- Orient solar panels and manage power / enter low-power sleep mode
- Re-plan route on detecting an obstacle or hazard

iv. Performance Measure

- Number and scientific quality/accuracy of samples correctly identified and analysed
- Total safe distance travelled and area of terrain explored/mapped
- Volume and accuracy of data successfully transmitted back to Earth
- Energy efficiency — power consumed relative to tasks accomplished
- Avoidance of damage, entrapment, or loss of the rover (safety)
- Time taken to achieve mission goals within the operational budget

v. Suitable Agent Architecture

A **hybrid, learning, model-based goal/utility-based agent** is best suited. Because the environment is partially observable and dynamic, the rover must maintain an internal model/state estimate of the world (model-based) rather than react to raw percepts alone. Because it must plan multi-step routes and select which rocks are worth sampling, it needs goal- and utility-based reasoning (weighing scientific value versus energy/time cost) rather than a simple table of rules. A purely reflex agent cannot plan ahead, and a purely deliberative planner is too slow to react to sudden hazards, so real rovers use a **layered/hybrid architecture**: a fast reactive layer for immediate obstacle avoidance, a deliberative planning layer for route and task planning using an internal map, and a learning component that improves terrain and hazard models over time from experience.

Question 3: 8-Queens Problem — Search Space Formulation

Goal: place 8 queens on an 8×8 chessboard so that no two queens attack each other (share a row, column, or diagonal). In the figure referred to in the question, two queens lie on the same diagonal (first column and last column), so that configuration is not a solution.

Problem Formulation (incremental / column-by-column formulation)

Component	Definition for 8-Queens
States	Any arrangement of 0 to 8 queens on the board, with at most one queen per column, placed only in
Initial state	Empty board — no queens placed.
Actions / Successor function	Add a queen to the left-most empty column, in any row of that column that is not attacked by an exist
Goal test	8 queens are on the board and no queen attacks another (no shared row / column / diagonal).
Path cost	Not relevant — every action costs 1; only the final configuration matters, not the path taken to reach

Search Space Size

- Naive formulation (any queen on any square): $C(64, 8) \approx 4.4 \times 10^9$ states.
- One queen per column, any row: $8^8 = 16,777,216$ states.
- One queen per row and column (permutation formulation): $8! = 40,320$ states — the row/diagonal check then eliminates non-solutions.

Choice of Search Strategy

Since the path to a solution is irrelevant (only the final board configuration matters), 8-Queens is best explored with algorithms that don't track path cost: **depth-first / backtracking search** (place a queen per column, backtrack when a column has no safe row) is a common complete solver, while **local search** methods such as hill-climbing with random restarts or the min-conflicts heuristic are very effective in practice because they start from a complete assignment of 8 queens (one per column) and repeatedly move the queen with the most conflicts to reduce attacks, which solves even the million-queens version almost instantly.

Question 4: OLA Cab Booking — Problem-Solving Agent

A customer books a ride from a source to a destination and picks a cab category (Mini, Micro, Sedan, Shared, Prime, etc.) based on comfort/cost preference.

Type of Agent

This is best modelled as a **goal-based problem-solving agent** (a simple problem-solving/search agent). The agent's goal is well-defined — get the customer from the pickup location to the destination using the chosen cab type at acceptable cost/time — and it uses a search process over possible routes and available cabs to find a plan that reaches that goal. It can further be seen as a **utility-based agent** when it must rank multiple candidate cabs/routes by a combination of fare, ETA, and cab type preference rather than just any goal-satisfying option.

PEAS (brief)

- **Performance:** ride booked correctly, minimum wait time and fare, driver reaches on time, customer rating
- **Environment:** city road network, live traffic, GPS maps, available drivers, pricing engine
- **Actuators:** booking confirmation, dispatch request to driver app, fare display, notifications
- **Sensors:** customer's pickup/drop GPS input, selected cab type, live driver locations, traffic data

Pseudocode

```
FUNCTION OLA_CAB_AGENT(pickup_location, drop_location, cab_type, customer_prefs
):
    INITIAL_STATE    <- current_location = pickup_location, cab_booked = FALSE
    GOAL_STATE       <- current_location = drop_location, cab_booked = TRUE, tri
p_completed = TRUE
    ACTIONS          <- {SelectCabType, SearchNearbyDrivers, ConfirmBooking,
        NavigateRoute(A* / Dijkstra on road network), PickupCu
stomer, DropCustomer}

    // 1. Formulate the problem
    available_cabs <- GET_NEARBY_CABS(pickup_location, cab_type)
    IF available_cabs is EMPTY:
        RETURN "No cabs available, try again"

    // 2. Search for the best option (utility-based ranking)
    FOR each cab IN available_cabs:
        cab.utility <- W1*(1/ETA) + W2*(1/fare) + W3*driver_rating
    best_cab <- ARGMAX(available_cabs, cab.utility)

    // 3. Formulate route as a search problem and solve it
    route <- SEARCH(ROAD_NETWORK, pickup_location, drop_location, algorithm = A
_STAR)

    // 4. Execute plan
    CONFIRM_BOOKING(best_cab, cab_type, route)
    NOTIFY(customer, best_cab.driver, ETA)
    WHILE current_location != drop_location:
        MOVE_ALONG(route)
        UPDATE(current_location)
    trip_completed <- TRUE
```

```
CHARGE_FARE(customer, computed_fare)
RETURN "Trip completed successfully"
```

Question 5: Uniform Cost Search on the Logistics Network

The delivery network (from the given graph) has warehouses S, A, B, C, D, G with the following directed weighted edges (cost = fuel + time):

Edge	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

Uniform Cost Search always expands the node in the frontier with the lowest cumulative path cost $g(n)$ from S, and only tests for the goal when a node is popped/expanded (not when it is generated), guaranteeing an optimal solution for non-negative edge costs.

Step-by-Step Trace

Step	Node Expanded	$g(n)$	Frontier after expansion (node : g)
1	S	0	A:1, G:12
2	A	1	C:2, G:12, B:4
3	C	2	D:3, G:4 (via S-A-C-G, replaces 12), B:4
4	D	3	G:4 (unchanged — S-A-C-D-G costs 6 > 4, discarded), B:4
5	G (goal)	4	Goal test succeeds — search stops

Alternative Paths Considered (for comparison)

Path	Total Cost
S → A → C → G (optimal)	1 + 1 + 2 = 4
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G (direct)	12

Result: The least-cost path from S to G is **S → A → C → G** with a total cost of **4** (fuel + time units), found and proven optimal by Uniform Cost Search.